Année de Formation 2025/2026

M205 : Développer en back end
TP : Manipulation des Vues

1. Création d'une page d'accueil avec Blade

Objectif : Créer une page d'accueil dynamique qui affiche un message de bienvenue.

1. Dans le répertoire `resources/views`, créez un fichier `welcome.blade.php`.
2. Ajoutez-y du HTML de base et intégrez une variable `$nom` avec Blade.
3. Dans le contrôleur (ou directement dans les routes pour simplifier), envoyez une variable `$nom` à la vue avec un nom fictif.
4. Affichez le message de bienvenue sur la page.

routes/web.php :

```php
Route::get('/', function () {
    return view('welcome', ['nom' => 'Alice']);
});
```

resources/views/welcome.blade.php :

```blade
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Accueil</title>
</head>
<body>
<h1>Bienvenue, {{ $nom }} !</h1>
</body>
</html>
```

2. Utilisation de la directive `@foreach` pour afficher une liste d'articles

Objectif : Créer une vue qui affiche une liste d'articles à partir d'un tableau transmis depuis le contrôleur.

1. Créez une nouvelle route qui passe une liste d'articles sous forme de tableau à la vue.
2. Utilisez `@foreach` dans le fichier Blade pour afficher chaque article.

routes/web.php :

```php
Route::get('/articles', function () {
    $articles = [
```

```
        ['titre' => 'Introduction à Laravel', 'auteur' => 'Alice'],
        ['titre' => 'Les bases de PHP', 'auteur' => 'Bob'],
        ['titre' => 'Guide du développeur web', 'auteur' => 'Clara'],
    ];
    return view('articles', ['articles' => $articles]);
});
```

resources/views/articles.blade.php :

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Articles</title>
</head>
<body>
<h1>Liste des articles</h1>
<ul>
@foreach($articles as $article)
<li><strong>{{ $article['titre'] }}</strong> par {{ $article['auteur'] }}</li>
@endforeach
</ul>
</body>
</html>
```

3. Création d'une vue avec une structure de mise en page Blade

Objectif : Utiliser le système de layouts de Blade pour créer une structure de base partagée par plusieurs pages.

1. Créez un fichier `layouts/app.blade.php` dans le dossier `views/layouts`.
2. Ajoutez-y une structure de page HTML de base avec un espace réservé `@yield` pour le contenu.
3. Créez ensuite deux fichiers de vue (`home.blade.php` et `about.blade.php`) qui héritent de ce layout.

resources/views/layouts/app.blade.php :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>@yield('title')</title>
</head>
<body>
    <header>
        <h1>Mon Application Laravel</h1>
        <nav>
            <a href="/">Accueil</a> | <a href="/about">À propos</a>
        </nav>
    </header>
    <main>
        @yield('content')
```

```
    </main>
    <footer>
        <p>&copy; 2024 Mon Application</p>
    </footer>
</body>
</html>
```

resources/views/home.blade.php :

```
@extends('layouts.app')

@section('title', 'Accueil')

@section('content')
<h2>Bienvenue sur la page d'accueil</h2>
<p>Ceci est le contenu de la page d'accueil.</p>
@endsection
```

resources/views/about.blade.php :

```
@extends('layouts.app')

@section('title', 'À propos')

@section('content')
<h2>À propos de notre application</h2>
<p>Cette application est un exemple simple sous Laravel 11.</p>
@endsection
```

4. Utilisation des composants Blade

Objectif : Créer un composant réutilisable pour un bouton, puis l'utiliser dans différentes pages.

1. Dans `resources/views/components`, créez un fichier `button.blade.php`.
2. Ajoutez un code HTML pour un bouton stylé et une variable `$label` pour le texte.
3. Utilisez le composant dans d'autres vues avec `<x-button label="..." />`.

resources/views/components/button.blade.php :

```
<button style="padding: 10px 20px; background-color: #4CAF50; color: white;">
{{ $label }}
</button>
```

home.blade.php:

```
<x-button label="Cliquez ici" />
<x-button label="En savoir plus" />
```

5. Utilisation de la façade View pour afficher des détails d'un produit

Objectif : Utiliser la façade `View` pour :

1. Vérifier l'existence d'une vue.
2. Partager des données globales avec toutes les vues.
3. Rendre une vue en envoyant des données dynamiques.

Étapes

1. Créer une vue pour les détails d'un produit

Créez une vue dans `resources/views/product.blade.php` qui affiche les informations d'un produit.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Détails du produit</title>
</head>
<body>
    <h1>Détails du produit</h1>
    <p><strong>Nom :</strong> {{ $product['nom'] }}</p>
    <p><strong>Description :</strong> {{ $product['description'] }}</p>
    <p><strong>Prix :</strong> {{ $product['prix'] }} €</p>
    <p><strong>Message global :</strong> {{ $globalMessage }}</p>
</body>
</html>
```

2. Ajouter une route pour afficher le produit

Dans `routes/web.php`, créez une route `/produit` qui utilise la façade `View` pour gérer la vue.

```
use Illuminate\Support\Facades\View;
Route::get('/produit', function () {
    // Exemple de données d'un produit
    $product = [
        'nom' => 'Ordinateur portable',
        'description' => 'Un ordinateur portable performant pour le développement.',
        'prix' => 1200,
    ];
    // Partager un message global avec toutes les vues
    View::share('globalMessage', 'Ce message est visible sur toutes les pages.');

    // Vérifier si la vue "product" existe avant de la rendre
    if (View::exists('product')) {
        return View::make('product')->with('product', $product);
    } else {
        abort(404, 'La vue du produit n\'existe pas.');
    }
});
```